

使用 Cloud Workstations 和 NodeJS 進行 InnerLoop 開發

來源：<https://codelabs.developers.google.com/innerloop-dev-cloud-workstations-nodejs?hl=zh-tw>

步驟數：7

瞭解專為在容器化環境中透過 Cloud Workstations 開發 Nodejs 應用程式的軟體工程師所設計的功能與功能，簡化開發工作流程。

目錄

1. [1. 總覽](#)
2. [2. 設定和需求](#)
3. [3. 建立新的 Node.js 啟動應用程式](#)
4. [4. 建立新的入門應用程式](#)
5. [5. 逐步完成開發程序](#)
6. [6. 開發簡易的 CRUD REST 服務](#)
7. [7. 清除所用資源](#)

步驟 1 · 約 0 分鐘

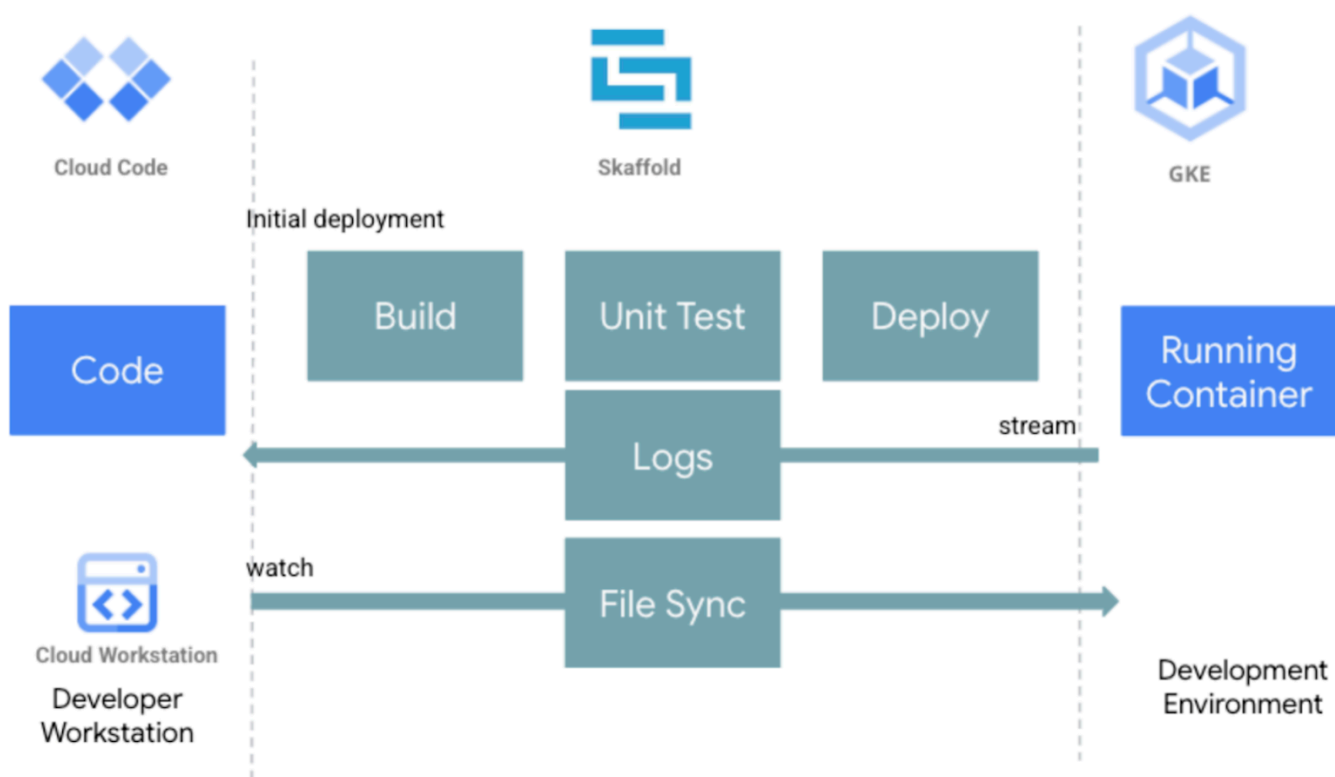
1. 總覽

本實驗室將展示各項功能，協助軟體工程師簡化開發工作流程，在容器化環境中開發 NodeJS 應用程式。一般容器開發作業需要使用者瞭解容器的詳細資料和容器建構程序。此外，開發人員通常必須中斷流程，離開 IDE 在遠端環境中測試及偵錯應用程式。有了本教學課程中提及的工具和技術，開發人員就能在 IDE 中有效處理容器化應用程式。

學習目標

在本實驗室中，您將瞭解在 Google Cloud 中使用容器進行開發的方法，包括：

- 建立入門 Node.js 應用程式
- 設定 Node.js 應用程式以進行容器開發
- 編寫簡單的 CRUD REST 服務
- 部署至 GKE
- 偵錯錯誤狀態
- 運用中斷點 / 記錄
- 將變更熱部署回 GKE
- 選用：整合 CloudSQL，做為後端持續性儲存空間



2. 設定和需求

自修實驗室環境設定

1. 登入 [Google Cloud 控制台](#)，然後建立新專案或重複使用現有專案。如果沒有 Gmail 或 Google Workspace 帳戶，請先[建立帳戶](#)。

The screenshot shows the Google Cloud Platform interface. At the top, there's a blue header with the Google Cloud Platform logo and a 'Select a project' button, which is circled in blue. Below this, the 'Select a project' section includes a search bar. The 'New Project' section follows, with a 'Project name' field containing 'my-kubernetes-codelab', also circled in blue. Below the name field, the 'Project ID' is displayed as 'my-kubernetes-codelab' with a note that it cannot be changed later. The 'Location' is set to 'No organization'. At the bottom, there are 'CREATE' and 'CANCEL' buttons.

- **專案名稱**是這個專案參與者的顯示名稱。這是 Google API 未使用的字元字串。你隨時可以更新該位置資訊。
- **專案 ID** 在所有 Google Cloud 專案中都是不重複的，而且設定後即無法變更。Cloud 控制台會自動產生不重複的字串，通常您不需要在意這個字串。在大多數程式碼研究室中，您需要參照專案 ID (通常會標示為 `PROJECT_ID`)。如果您不喜歡產生的 ID，可以產生另一個隨機 ID。你也可以嘗試使用自己的名稱，看看是否可用。完成這個步驟後就無法變更，且專案期間都會維持這個設定。
- 請注意，部分 API 會使用第三個值，也就是「專案編號」。如要進一步瞭解這三種值，請參閱[說明文件](#)。

注意：專案 ID 在全域中是專屬 ID，選定後就無法變更，且其他使用者無法使用。您是該 ID 的唯一使用者。即使刪除專案，該 ID 也無法再使用。

注意：如果您使用 Gmail 帳戶，可以將預設位置設為「沒有機構」。如果您使用 Google Workspace 帳戶，請選擇適合貴機構的位置。

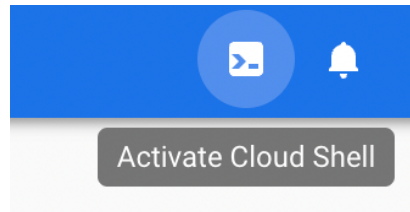
2. 接著，您需要在 Cloud 控制台中[啟用帳單](#)，才能使用 Cloud 資源/API。完成本程式碼研究室的費用應該不高，甚至完全免費。如要關閉資源，避免產生本教學課程以外的費用，您可以刪除自己建立的資源，或刪除整個專案。Google

Cloud 新使用者可參加[價值\\$300 美元的免費試用計畫](#)。

啟動 Cloud Shell 編輯器

本實驗室專為 Google Cloud Shell 編輯器設計及測試，如要存取編輯器，請按照下列步驟操作：

1. 前往 <https://console.cloud.google.com> 存取 Google 專案。
2. 按一下右上角的 Cloud Shell 編輯器圖示



3. 視窗底部會開啟新窗格
4. 按一下「開啟編輯器」按鈕



5. 編輯器會開啟，右側顯示檔案總管，中央區域則顯示編輯器
6. 畫面底部也應顯示終端機窗格
7. 如果終端機尚未開啟，請使用 `ctrl+` 鍵組合開啟新的終端機視窗

設定 gcloud

在 Cloud Shell 中，設定專案 ID 和要部署應用程式的區域。分別儲存為 `PROJECT_ID` 和 `REGION` 變數。

```
export PROJECT_ID=$(gcloud config get-value project)
export PROJECT_NUMBER=$(gcloud projects describe $PROJECT_ID --format='value(projectNumber)')
```

佈建本實驗室使用的基礎架構

在本實驗室中，您將程式碼部署至 GKE，並存取儲存在 Cloud SQL 資料庫中的資料。下方的設定指令碼會為您準備好基礎架構。

1. 下載設定指令碼並設為可執行。

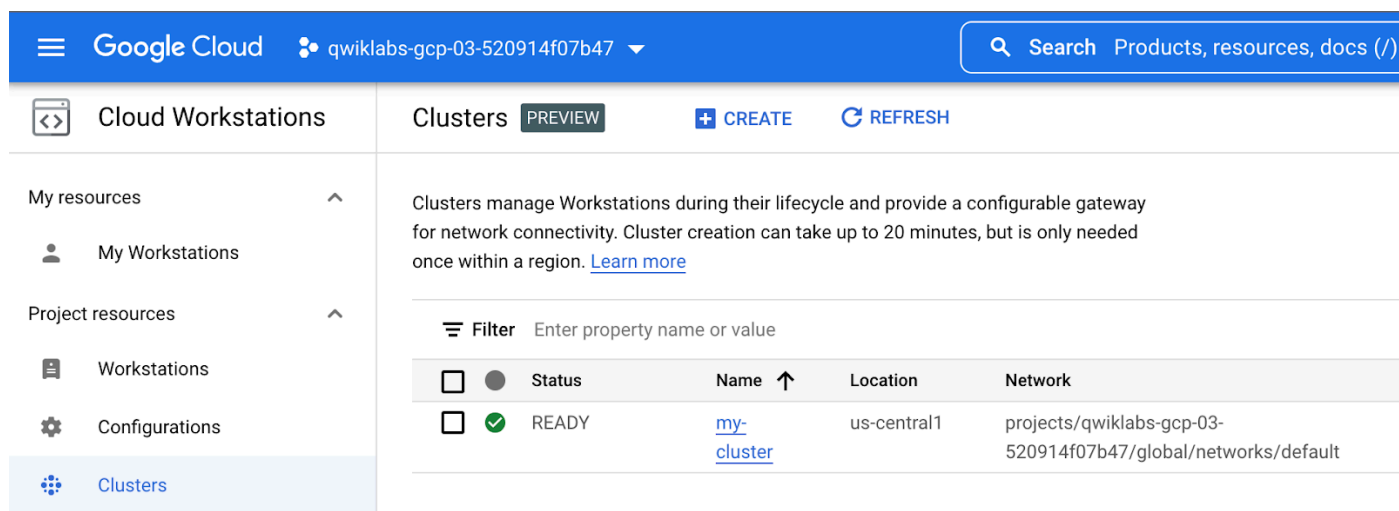
```
wget https://raw.githubusercontent.com/GoogleCloudPlatform/container-developer-workshop/main/labs/nodejs/setup_with_cw.sh
chmod +x setup_with_cw.sh
```

2. 開啟 `setup_with_cw.sh` 檔案，並編輯目前設為 CHANGEME 的密碼值
3. 執行設定指令碼，建立本實驗室中使用的 GKE 叢集和 Cloud SQL 資料庫

```
./setup_with_cw.sh &
```

Cloud Workstations 叢集

1. 在 Cloud 控制台中開啟 [Cloud Workstations](#)。等待叢集處於 **READY** 狀態。



The screenshot shows the Google Cloud Cloud Workstations interface. The left sidebar has a menu with 'My resources' (My Workstations), 'Project resources' (Workstations, Configurations, Clusters), and 'Clusters' (selected). The main content area is titled 'Clusters' with a 'PREVIEW' tab, '+ CREATE' button, and 'REFRESH' button. It contains a description: 'Clusters manage Workstations during their lifecycle and provide a configurable gateway for network connectivity. Cluster creation can take up to 20 minutes, but is only needed once within a region. [Learn more](#)'. Below this is a filter bar and a table with one row showing a cluster in 'READY' status.

Status	Name	Location	Network
READY	my-cluster	us-central1	projects/qwiklabs-gcp-03-520914f07b47/global/networks/default

建立工作站設定

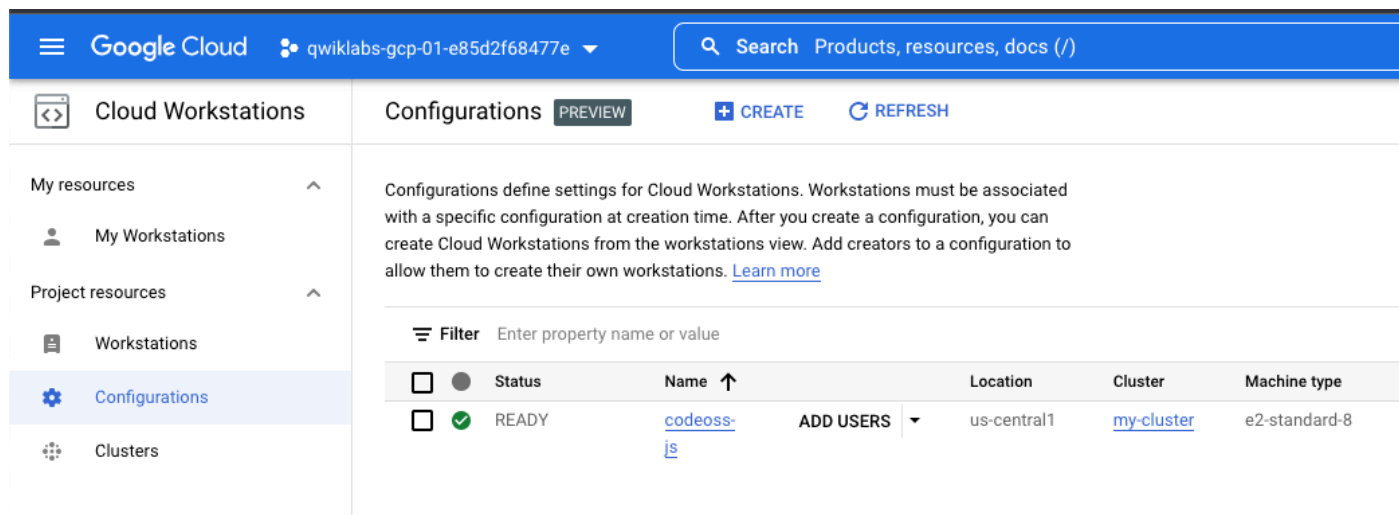
1. 如果 Cloud Shell 工作階段已中斷連線，請按一下「重新連線」，然後執行 `gcloud` 指令列指令來設定專案 ID。執行指令前，請將下方的範例專案 ID 換成您的 Qwiklabs 專案 ID。

```
gcloud config set project qwiklabs-gcp-project-id
```

2. 在終端機下載並執行下列指令碼，建立 Cloud Workstations 設定。

```
wget https://raw.githubusercontent.com/GoogleCloudPlatform/container-developer-workshop/main/labs/nodejs/workstation_config_setup.sh
chmod +x workstation_config_setup.sh
./workstation_config_setup.sh
```

3. 在「設定」部分下方驗證結果。轉換為「READY」狀態需要 2 分鐘。



The screenshot shows the Google Cloud Cloud Workstations interface, specifically the 'Configurations' page. The left sidebar has a menu with 'My resources' (My Workstations), 'Project resources' (Workstations, Configurations, Clusters), and 'Configurations' (selected). The main content area is titled 'Configurations' with a 'PREVIEW' tab, '+ CREATE' button, and 'REFRESH' button. It contains a description: 'Configurations define settings for Cloud Workstations. Workstations must be associated with a specific configuration at creation time. After you create a configuration, you can create Cloud Workstations from the workstations view. Add creators to a configuration to allow them to create their own workstations. [Learn more](#)'. Below this is a filter bar and a table with one row showing a configuration in 'READY' status.

Status	Name	Location	Cluster	Machine type
READY	codeooss-is	us-central1	my-cluster	e2-standard-8

4. 在控制台中開啟 [Cloud Workstations](#)，然後建立新執行個體。

← → ↻ console.cloud.google.com/workstations/list?project=qwiklabs-gcp-01-799f5ac5dfc5

Google Cloud qwiklabs-gcp-01-799f5ac5dfc5 Search workstation

Cloud Workstations Workstations PREVIEW + CREATE REFRESH

My resources ^

- My Workstations

Project resources ^

- Workstations
- Configurations
- Clusters

5. 將名稱變更為 `my-workstation`，然後選取現有設定：`codeoss-js`。

Google Cloud qwiklabs-gcp-01-e85d2f68477e Search Products, resources, docs (/)

Cloud Workstations Create workstation PREVIEW

My resources ^

- My Workstations

Project resources ^

- Workstations
- Configurations
- Clusters

Name *
my-workstation

Configuration *
codeoss-js (e2-standard-8)

6. 驗證「工作站」部分下的結果。

Google Cloud qwiklabs-gcp-01-e85d2f68477e Search Products, resources, docs (/)

Cloud Workstations Workstations PREVIEW + CREATE REFRESH

My resources ^

- My Workstations

Project resources ^

- Workstations
- Configurations
- Clusters

Cloud Workstations provides managed, on-demand, development environments in the cloud. They can be accessed through a browser UI, a terminal/SSH, or from your local IDE through an SSH bridge. [Learn more](#)

Filter Enter property name or value

☐	Status	Name	Location	Cluster	Configuration	Machine type
☐	Stopped	my-workstation	us-central1	my-cluster	codeoss-js	e2-standard-8

啟動工作站

1. 啟動工作站。工作站會在幾分鐘內啟動。

Google Cloud

qwiklabs-gcp-01-e85d2f68477e

Search Products, resources, docs (/)

Cloud Workstations

My resources

My Workstations

Project resources

Workstations

Configurations

Clusters

Workstations

PREVIEW

+ CREATE

REFRESH

Cloud Workstations provides managed, on-demand, development environments in the cloud. They can be accessed through a browser UI, a terminal/SSH, or from your local IDE through an SSH bridge. [Learn more](#)

Filter Enter property name or value

Status	Name	Location	Cluster	Configuration	Machine type
Running	my-workstation	us-central1	my-cluster	codeoss-js	e2-standard-8

2. 按一下網址列中的圖示，允許第三方 Cookie。

Workstations – Cloud Worksta

Get Started — Code OSS for Cl

80-my-workstation.cluster-ixavuiopfdfs224plj24ejzm.cloudworkstations.dev/?authuser=0

Incognito

Third-party cookie blocking

Get Started

3. 按一下「網站無法正常運作嗎？」。

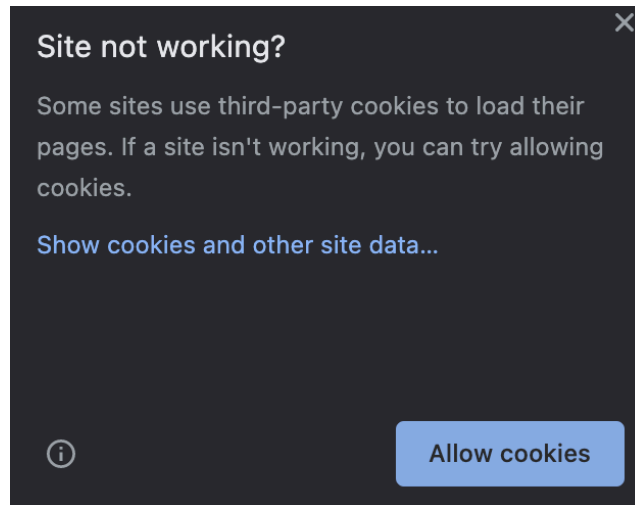
m.cloudworkstations.dev/?authuser=0

6 third-party cookies are blocked

Sites can't use cookies that track you across the web

Site not working?

4. 按一下「允許 Cookie」。



5. 工作站啟動後，您會看到 Code OSS IDE。在工作站 IDE 的「開始使用」頁面中，按一下「標示為完成」

[< Get Started](#)



Get Started with Code OSS in the Web

Discover the best customizations to make Code OSS in the Web yours.

☐ Choose the look you want

The right color palette helps you focus on your code, is easy on your eyes, and is simply more fun to use.

[Browse Color Themes](#)

Tip: Use keyboard shortcut (**⌘K ⌘T**)

☐ One shortcut to access everything

☐ Just the right amount of UI

☐ Rich support for all your languages

☒ Mark Done

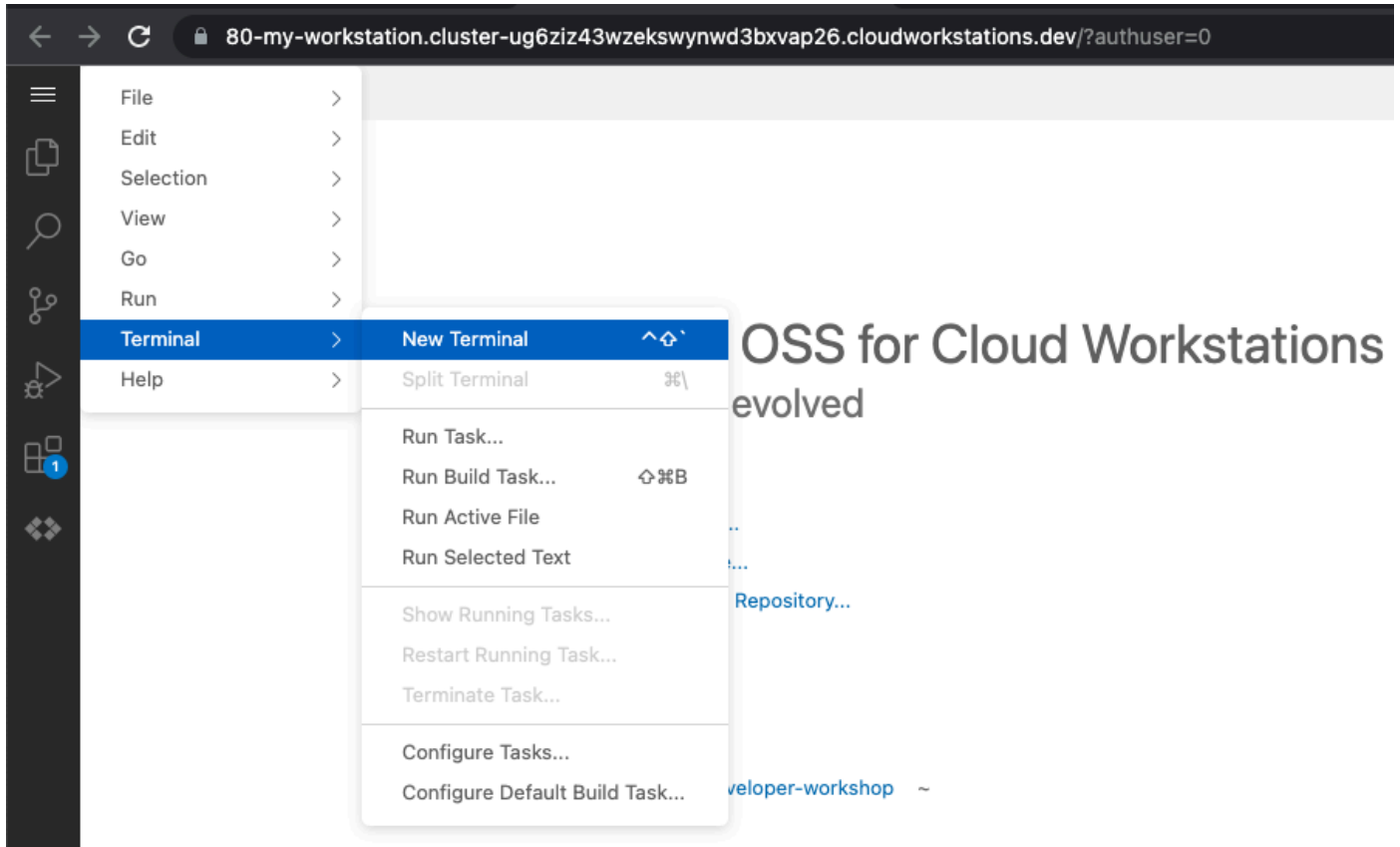
[Next Section →](#)

步驟 3 · 約 0 分鐘

3. 建立新的 Node.js 啟動應用程式

在本節中，您將建立新的 Node.js 應用程式。

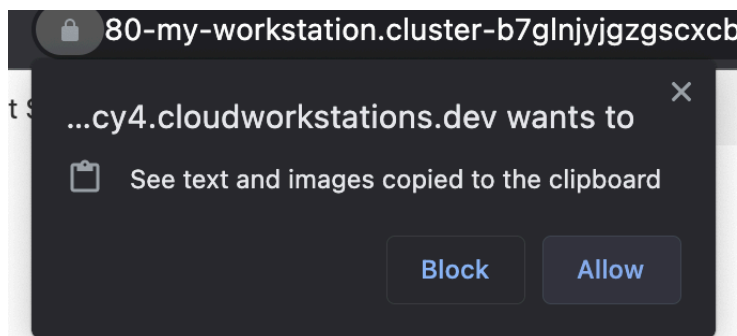
1. 開啟新的終端機。



2. 在 Cloud Shell 中，建立名為 `mynodejsapp` 的新目錄

```
mkdir mynodejsapp
```

如果看到這則訊息，請按一下「允許」按鈕，這樣就能複製並貼到工作站。



3. 切換至這個目錄，然後開啟做為工作區。這會在新建的資料夾中建立工作區設定，重新載入編輯器。

```
cd mynodejsapp && code-oss-cloud-workstations -r --folder-uri="$PWD"
```

1. 再次開啟新的終端機。使用 NVM 安裝 Node 和 NPM。

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.2/install.sh | bash
```

```
    # This loads nvm bash_completion
export NVM_DIR="$HOME/.nvm"
[ -s "$NVM_DIR/nvm.sh" ] && \. "$NVM_DIR/nvm.sh" # This loads nvm
[ -s "$NVM_DIR/bash_completion" ] && \. "$NVM_DIR/bash_completion"

nvm install stable

nvm alias default stable
```

步驟 4 · 約 0 分鐘

4. 建立新的入門應用程式

1. 初始化應用程式

執行下列指令，建立 `package.json` 檔案

```
npm init
```

Choose the ``entry point: (index.js) src/index.js`` and leave default values for the rest of the parameters. This will create the ``package.json`` file with following contents

```
{
  "name": "mynodejsapp",
  "version": "1.0.0",
  "description": "",
  "main": "src/index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "author": "",
  "license": "ISC"
}
```

2. 新增進入點

在 IDE 中開啟並編輯 `package.json` 檔案，在指令碼 `"start": "node src/index.js"`，中加入啟動指令。變更後，指令碼應如下列程式碼片段所示：

```
"scripts": {
  "start": "node src/index.js",
  "test": "echo \"Error: no test specified\" && exit 1"
},
```

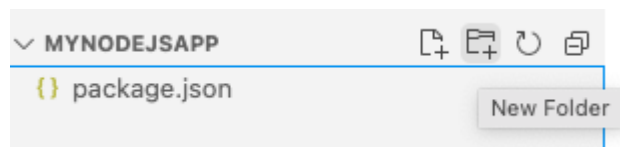
3. 新增 Express 依附元件

我們要新增的程式碼也會使用 `express`，因此請將該依附元件新增至這個 `package.json` 檔案。因此，完成所有變更後，`package.json` 檔案應如下所示。

```
{
  "name": "mynodejsapp",
  "version": "1.0.0",
  "description": "",
  "main": "src/index.js",
  "scripts": {
    "start": "node src/index.js",
    "test": "echo \\\"Error: no test specified\\\" && exit 1"
  },
  "author": "",
  "license": "ISC",
  "dependencies": {
    "express": "^4.17.3"
  }
}
```

4. 建立 index.js 檔案

在檔案總管檢視畫面中選取「New Folder」，建立名為 src 的來源目錄。



建立 src/index.js 檔案



使用下列程式碼

```
const express = require('express');
const app = express();
const PORT = 8080;

app.get('/', (req, res) => {
  var message="Greetings from Node";
  res.send({ message: message });
});

app.listen(PORT, () => {
  console.log(`Server running at: http://localhost:${PORT}/`);
});
```

請注意，通訊埠 已設為值 8080

產生資訊清單

Scaffold 提供整合式工具，可簡化容器開發作業。在這個步驟中，您將初始化 Scaffold，系統會自動建立基本 Kubernetes YAML 檔案。執行下列指令，開始進行程序。

在終端機中執行下列指令

```
scaffold init --generate-manifests
```

出現提示時：

- 輸入 **8080** 做為通訊埠
- 輸入 **y** 儲存設定

兩個檔案會新增至工作區，分別是 `scaffold.yaml` 和 `deployment.yaml`。

更新應用程式名稱

設定中包含的預設值目前與應用程式名稱不符。更新檔案，參照應用程式名稱，而非預設值。

1. 變更 Skaffold 設定中的項目

- 開啟「`scaffold.yaml`」
- 選取目前設為 `package-json-image` 的圖片名稱
- 按一下滑鼠右鍵，然後選擇「變更所有出現位置」
- 輸入新名稱，如 `mynodejsapp`

2. 變更 Kubernetes 設定中的項目

- 開啟 `deployment.yaml` 檔案
- 選取目前設為 `package-json-image` 的圖片名稱
- 按一下滑鼠右鍵，然後選擇「變更所有出現位置」
- 輸入新名稱，如 `mynodejsapp`

請注意，在 `scaffold.yaml` 檔案中，`build` 區段會使用 `buildpacks` 將應用程式容器化。這個程式碼沒有 `Dockerfile`，開發人員也不需要任何 `Docker` 知識，就能將這個應用程式容器化。

此外，透過這項 `skaffold` 設定，編輯器和執行中的容器之間也會自動啟用熱同步。啟用熱同步功能不需要額外設定。

步驟 5 · 約 0 分鐘

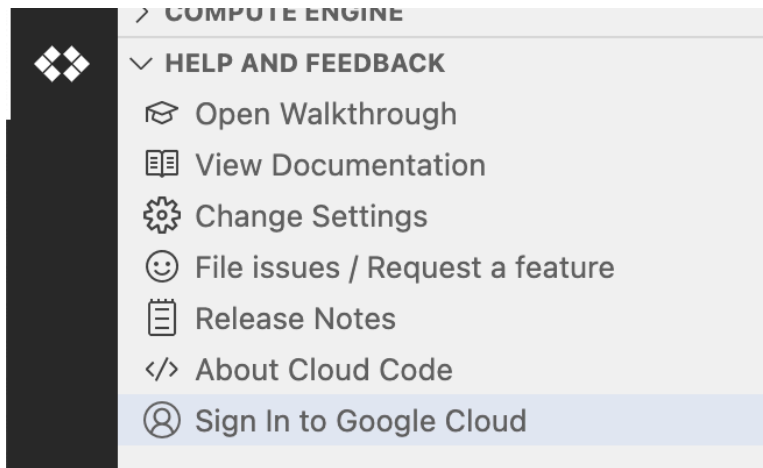
5. 逐步完成開發程序

在本節中，您將使用 Cloud Code 外掛程式逐步完成幾個步驟，瞭解基本程序，並驗證入門應用程式的設定。

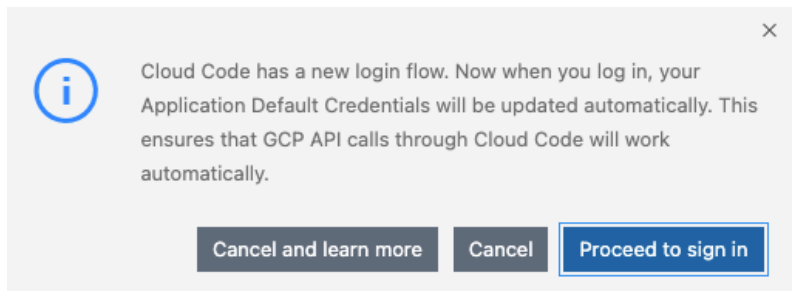
Cloud Code 整合了 Skaffold，可簡化開發程序。在後續步驟中部署至 GKE 時，Cloud Code 和 Skaffold 會自動建構容器映像檔、將其推送至 Container Registry，然後將應用程式部署至 GKE。這項作業會在幕後進行，將詳細資料從開發人員流程中抽象化。此外，Cloud Code 也為以容器為基礎的開發作業提供傳統的偵錯和熱同步功能，進一步提升開發流程。

登入 Google Cloud

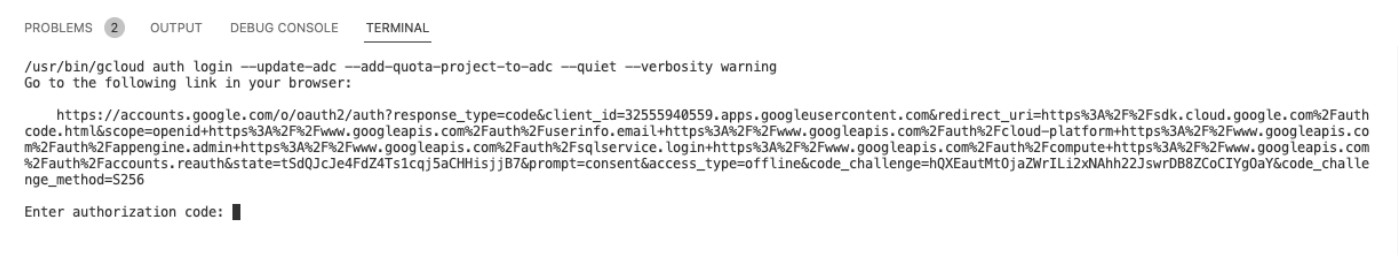
1. 按一下 Cloud Code 圖示，然後選取「Sign in to Google Cloud」(登入 Google Cloud)：



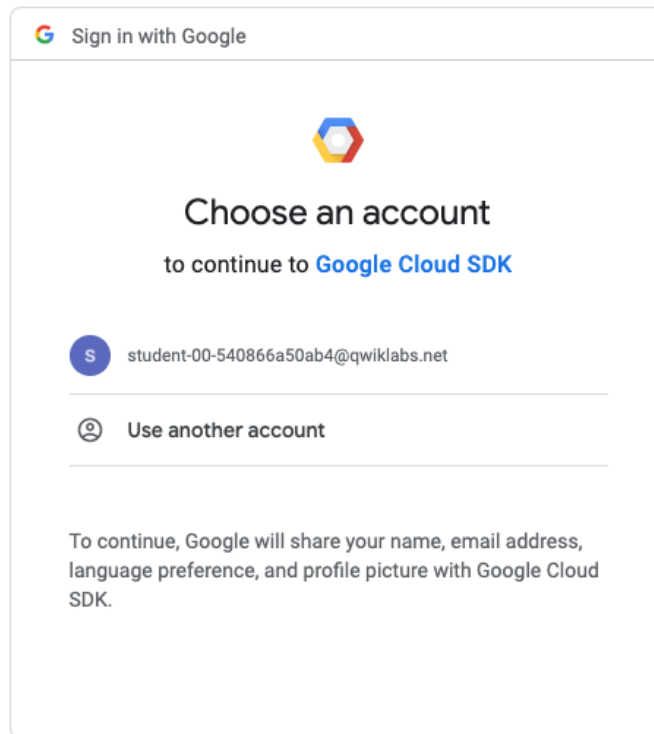
2. 按一下「Proceed to sign in」(繼續登入)。



3. 查看終端機的輸出內容並開啟連結：



4. 使用 Qwiklabs 學生憑證登入。



5. 選取「允許」：



Google Cloud SDK wants to access your Google Account

 student-00-540866a50ab4@qwiklabs.net

This will allow **Google Cloud SDK** to:

- See, edit, configure, and delete your Google Cloud data and see the email address for your Google Account. 
- View and sign in to your Google Cloud SQL instances 
- View and manage your Google Compute Engine resources 
- View and manage your applications deployed on Google App Engine 

Make sure you trust Google Cloud SDK

You may be sharing sensitive info with this site or app. You can always see or remove access in your [Google Account](#).

Learn how Google helps you [share data safely](#).

See Google Cloud SDK's [Privacy Policy](#) and [Terms of Service](#).

Cancel

Allow

6. 複製驗證碼，然後返回「工作站」分頁。



Sign in to the gcloud CLI

You are seeing this page because you ran the following command in the gcloud CLI from this or another machine. If this is not the case, close this tab.

```
gcloud auth login --no-launch-browser
```

Enter the following verification code in gcloud CLI on the machine you want to log into. This is a credential similar to your password and should not be shared with others.

```
4/0AfgeXvs8ORnkDVPb33lRxLZdqK7X8n6yAo  
EV53FBnUCkmTQuUaGEZXo0BWl4Z7U07wbyXg
```

Copy

You can close this tab when you're done.

7. 貼上驗證碼，然後按下 Enter 鍵。

```
/usr/bin/gcloud auth login --update-adc --add-quota-project-to-adc --quiet --verbosity warning  
Go to the following link in your browser:
```

```
https://accounts.google.com/o/oauth2/auth?response_type=code&client_id=32555940559.apps.googleusercontent.com&redirect_uri=https%3A%2F%2Fsdg.cloud.google.com%2Fauth  
code.html&scope=openid+https%3A%2F%2Fwww.googleapis.com%2Fauth%2Fuserinfo.email+https%3A%2F%2Fwww.googleapis.com%2Fauth%2Fcloud-platform+https%3A%2F%2Fwww.googleapis.co  
m%2Fauth%2Fappengine.admin+https%3A%2F%2Fwww.googleapis.com%2Fauth%2Fsqlservice.login+https%3A%2F%2Fwww.googleapis.com%2Fauth%2Fcompute+https%3A%2F%2Fwww.googleapis.com  
%2Fauth%2Faccounts.reauth&state=tSdQJcJe4FdZ4Ts1cqj5aCHHsjjB7&prompt=consent&access_type=offline&code_challenge=hQXEautMt0jaZwrIL12xNAhh22JswrDB8ZCoCIYg0aY&code_challe  
nge_method=S256
```

```
Enter authorization code: 4/0AfgeXvs8ORnkDVPb33lRxLZdqK7X8n6yAoEV53FBnUCkmTQuUaGEZXo0BWl4Z7U07wbyXg  
WARNING: Cannot find a project to insert into application default credentials (ADC) as a quota project.  
Run $gcloud auth application-default set-quota-project to insert a quota project to ADC.
```

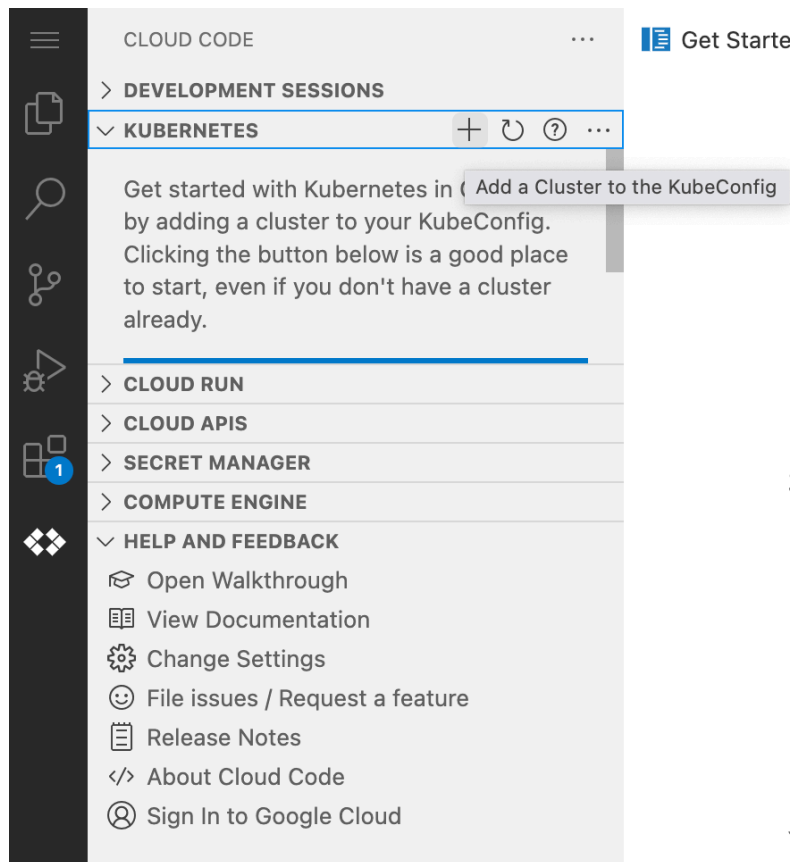
```
Application default credentials (ADC) were updated.
```

```
You are now logged in as [student-00-540866a50ab4@qwiklabs.net].  
Your current project is [None]. You can change this setting by running:  
$ gcloud config set project PROJECT_ID
```

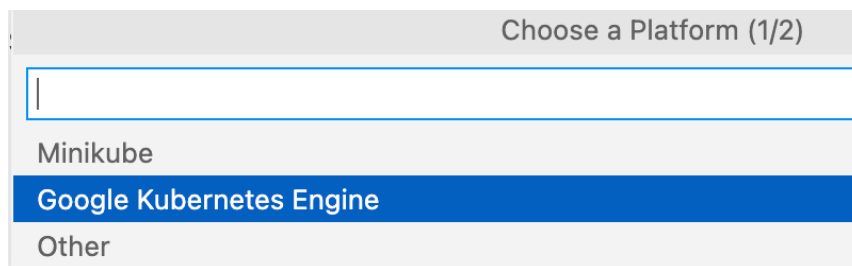
```
Press any key to exit.[]
```

新增 Kubernetes 叢集

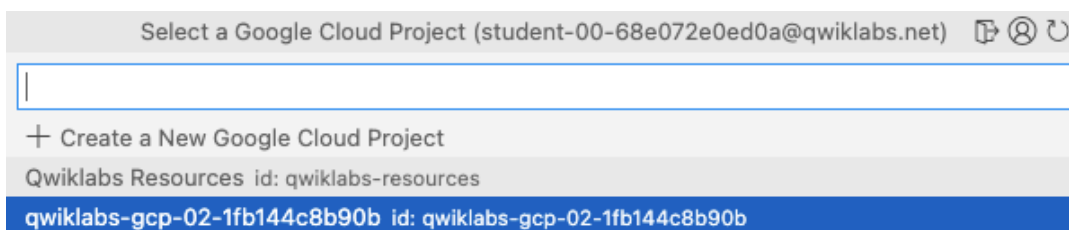
1. 新增叢集



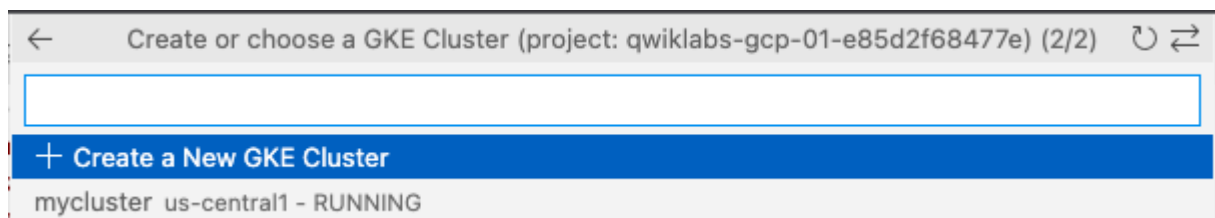
2. 選取 Google Kubernetes Engine :



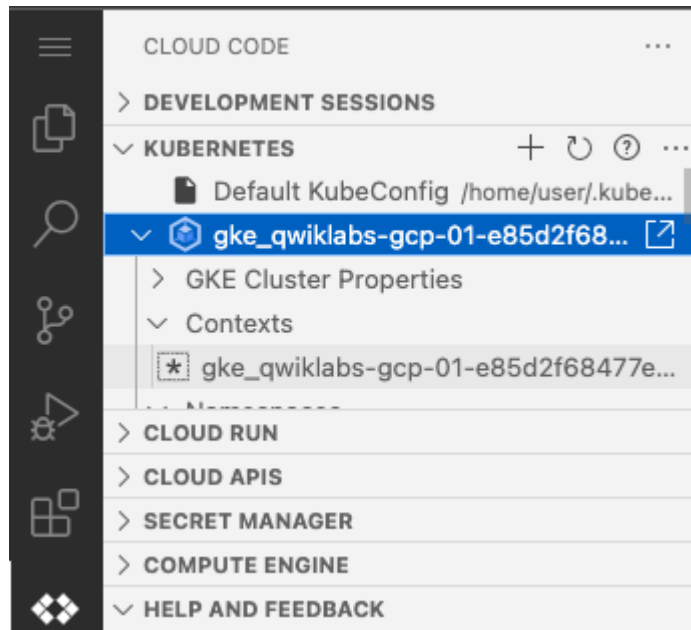
3. 選取專案。



4. 選取在初始設定中建立的「mycluster」。



5. 現在 Cloud Code 下方的 Kubernetes 叢集清單中會顯示該叢集。從這裡瀏覽及探索叢集。



使用 gcloud CLI 設定目前的專案 ID

1. 從 Qwiklabs 頁面複製這個實驗室的專案 ID。

Caution: When you are in the console, do not deviate from the lab instructions. Doing so may cause your account to be blocked. [Learn more.](#)

[Console](#)

Username

student-00-1f28f50e2fc3 

Password

CJu118bpNfZE 

Project ID

qwiklabs-gcp-04-d75bba0 

2. 在終端機中執行 gcloud CLI 指令，設定專案 ID。請先將範例專案 ID 改為對應的值，再執行指令。執行下列指令前，請先「取代」專案 ID。

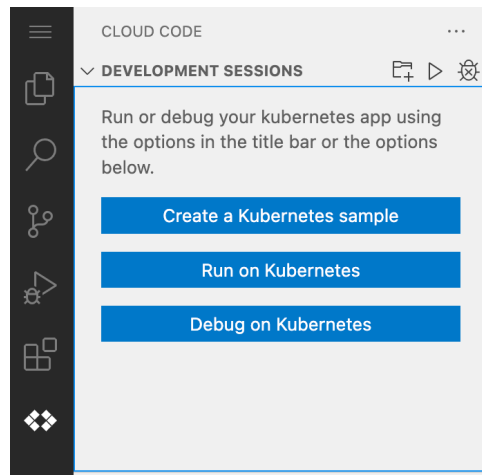
```
gcloud config set project qwiklabs-gcp-project-id
```

部署到 Kubernetes

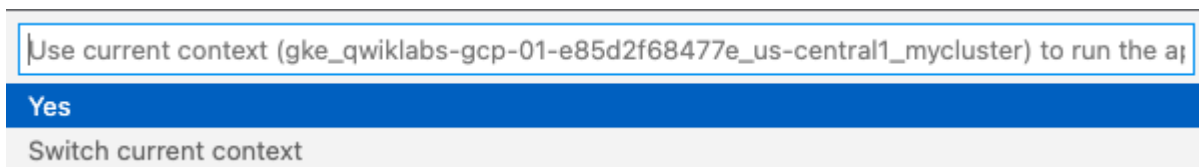
1. 在 Cloud Shell 編輯器底部的窗格中，選取 Cloud Code 



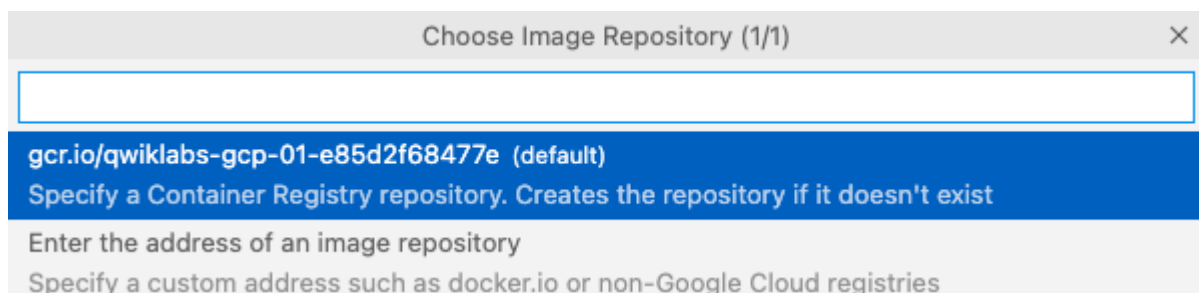
2. 在「DEVELOPMENT SESSIONS」下方顯示的面板中，選取「Run on Kubernetes」。如果系統顯示提示，請選取「Yes」使用目前的 Kubernetes 環境。



3. 首次執行指令時，畫面頂端會顯示提示，詢問您是否要使用目前的 Kubernetes 內容，請選取「Yes」接受並使用目前的內容。



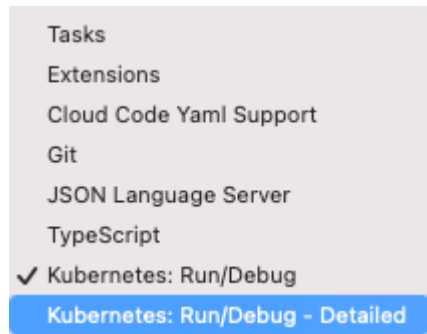
4. 接著系統會顯示提示，詢問要使用哪個容器登錄服務。按下 Enter 鍵接受預設值



5. 在下方窗格中選取「Output」分頁，然後在下拉式選單中選取「Kubernetes: Run/Debug」，即可查看進度 and 通知



6. 在右側的管道下拉式選單中選取「Kubernetes: Run/Debug - Detailed」，即可查看其他詳細資料和容器的即時記錄檔串流



7. 從下拉式選單選取「Kubernetes: Run/Debug」，即可返回簡化檢視畫面
8. 建構和測試完成後，「輸出」分頁會顯示 `Resource deployment/mynodejsapp status completed successfully`，並列出網址：「Forwarded URL from service demo-app: `http://localhost: 8080`」（從服務 demo-app 轉送的網址：`http://localhost:8080`）
9. 在 Cloud Code 終端機中，將游標懸停在輸出內容中的網址 (`http://localhost:8080`)，然後在顯示的工具提示中選取「Follow Link」。

回覆內容如下：

```
{"message": "Greetings from Node"}
```

熱重載

1. 前往 `src/index.js`。編輯問候訊息的程式碼，然後前往 `'Hello from Node'`

請注意，在 `Output` 視窗的 `Kubernetes: Run/Debug` 檢視畫面中，監控程式會將更新後的檔案與 `Kubernetes` 中的容器同步

```
Update initiated
File sync started for 1 files for
gcr.io/myproject/mynodejsapp:latest@sha256:f554756b3b4d6c301c4b26ef96102227cfa2833270
db56241248ae42baa1971a
File sync succeeded for 1 files for
gcr.io/myproject/mynodejsapp:latest@sha256:f554756b3b4d6c301c4b26ef96102227cfa2833270
db56241248ae42baa1971a
Update succeeded
```

2. 如果切換至 `Kubernetes: Run/Debug - Detailed` 檢視畫面，您會發現系統會辨識檔案變更並重新啟動節點

```
files modified: [src/index.js]
Copying files:map[src/index.js:
[/workspace/src/index.js]]to:gcr.io/myproject/mynodejsapp:latest@sha256:f554756b3b4d6c
301c4b26ef96102227cfa2833270db56241248ae42baa1971a
Syncing 1 files for
gcr.io/myproject/mynodejsapp:latest@sha256:f554756b3b4d6c301c4b26ef96102227cfa2833270
db56241248ae42baa1971a
Watching for changes...
[mynodejsapp]
[mynodejsapp]> mynodejsapp@1.0.0 start /workspace
[mynodejsapp]> node src/index.js
[mynodejsapp]
[mynodejsapp]Server running at: http://localhost:8080/
```

3. 重新整理瀏覽器即可查看更新後的結果。

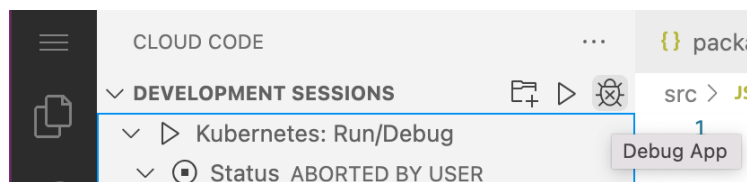
偵錯

1. 前往「偵錯」檢視畫面，然後停止目前的執行緒。



。

2. 按一下底部選單中的 **Cloud Code**，然後選取 **Debug on Kubernetes**，即可在 **debug** 模式下執行應用程式。



- 在 **Output** 視窗的 **Kubernetes Run/Debug - Detailed** 檢視畫面中，請注意 **skaffold** 會以偵錯模式部署這個應用程式。
- 應用程式會在幾分鐘內建構及部署完畢。這次您會發現偵錯工具已附加。

```
Port forwarding pod/mynodejsapp-6bbcf847cd-vqr6v in namespace default, remote port
9229 -> http://127.0.0.1:9229
[mynodejsapp]Debugger attached.
```

3. 底部的狀態列會從藍色變成橘色，表示目前處於偵錯模式。

⊗ 0 △ 0 🚀 Kubernetes: Run/Debug (mynodejsapp) </> Cloud Code ↔ qwiklabs-gcp-01-dac92f98ae1c

4. 在 **Kubernetes Run/Debug** 檢視畫面中，請注意已啟動可偵錯的容器

```
*****URLs*****
```

```
Forwarded URL from service mynodejsapp-service: http://localhost:8080
```

```
Debuggable container started pod/mynodejsapp-deployment-6bc7598798-xl9kj:mynodejsapp  
(default)
```

```
Update succeeded
```

```
*****
```

運用中斷點

1. 開啟 `src/index.js`
2. 找出顯示 `var message="Hello from Node";` 的陳述式
3. 點選行號左側的空白處，在該行新增中斷點。系統會顯示紅色指標，表示已設定中斷點
4. 重新載入瀏覽器，請注意偵錯工具會在該中斷點停止程序，並允許您調查在 GKE 中遠端執行的應用程式變數和狀態
5. 按一下變數部分，直到找到 `"message"` 變數為止。
6. 按下「Step over」(不進入)



執行該行

7. 觀察 `"message"` 變數的目前值是否變更為 `"Hello from Node"`
8. 按兩下變數名稱「target」，然後在彈出式視窗中將值變更為其他值，例如 `"Hi from Node"`
9. 按一下偵錯控制面板中的「繼續」按鈕
10. 在瀏覽器中查看回應，現在應該會顯示您剛輸入的更新值。
11. 按下停止按鈕



即可停止「偵錯」模式，再次點選中斷點即可移除。

6. 開發簡易的 CRUD REST 服務

此時，您的應用程式已完全設定為容器化開發，且您已透過 Cloud Code 逐步瞭解基本開發工作流程。在接下來的章節中，您將練習所學內容，新增連線至 Google Cloud 中代管資料庫的 REST 服務端點。

設定依附元件

應用程式程式碼會使用資料庫保存 REST 服務資料。在 `package.json` 檔案中加入下列內容，確保依附元件可用

1. 在 `package.json` 檔案中新增另外兩個依附元件 `pg` 和 `sequelize`，即可建構 CRUD 應用程式 Postgres。發布變更後，依附元件區段會如下所示。

```
"dependencies": {  
  "express": "^4.17.3",  
  "pg": "^8.8.0",  
  "sequelize": "^6.25.7"  
}
```

編寫 REST 服務的程式碼

1. 將 CRUD 應用程式程式碼新增至這個應用程式

```
wget -O app.zip https://github.com/GoogleCloudPlatform/container-developer-  
workshop/raw/main/labs/nodejs/app.zip  
  
unzip app.zip
```

這個程式碼有

- `models` 資料夾，其中包含 `item` 的實體模型
 - `controllers` 資料夾，內含執行 CRUD 作業的程式碼
 - `routes` 資料夾，可將特定網址模式轉送至不同呼叫
 - 包含資料庫連線詳細資料的 `config` 資料夾
2. 請注意，`db.config.js` 檔案中的資料庫設定是指連線至資料庫時需要提供的環境變數。此外，您也需要剖析傳入要求，瞭解網址編碼。
 3. 在 `src/index.js` 中新增下列程式碼片段，即可在以 `app.listen(PORT, () => {` 開頭的最後一個區段之前，從主要 JavaScript 檔案連線至 CRUD 程式碼。

```

const bodyParser = require('body-parser')
app.use(bodyParser.json())
app.use(
  bodyParser.urlencoded({
    extended: true,
  })
)
const db = require("../app/models");
db.sequelize.sync();
require("../app/routes/item.routes")(app);

```

4. 在 `deployment.yaml` 檔案中編輯部署作業，新增環境變數來提供資料庫連線資訊。

更新檔案結尾的規格項目，使其符合下列定義

```

spec:
  containers:
  - name: mynodejsapp
    image: mynodejsapp
    env:
    - name: DB_HOST
      value: ${DB_INSTANCE_IP}
    - name: DB_PORT
      value: "5432"
    - name: DB_USER
      valueFrom:
        secretKeyRef:
          name: gke-cloud-sql-secrets
          key: username
    - name: DB_PASS
      valueFrom:
        secretKeyRef:
          name: gke-cloud-sql-secrets
          key: password
    - name: DB_NAME
      valueFrom:
        secretKeyRef:
          name: gke-cloud-sql-secrets
          key: database

```

5. 將 `DB_HOST` 值替換為資料庫的位址

```

export DB_INSTANCE_IP=$(gcloud sql instances describe mytest-instance \
  --format=json | jq \
  --raw-output ".ipAddresses[].ipAddress")

envsubst < deployment.yaml > deployment.new && mv deployment.new deployment.yaml

```

部署及驗證應用程式

1. 在 Cloud Shell 編輯器底部的窗格中，選取 `Cloud Code`，然後選取畫面頂端的 `Debug on Kubernetes`。
2. 建構和測試完成後，「輸出」分頁會顯示 `Resource deployment/mynodejsapp status completed successfully`，並列出網址：「Forwarded URL from service mynodejsapp: http://localhost: 8080」（從服務 mynodejsapp 轉送的網址：http://localhost:8080）
3. 新增幾個項目。

在 Cloud Shell 終端機執行下列指令

```
URL=localhost:8080
curl -X POST $URL/items -d '{"itemName":"Body Spray", "itemPrice":3.2}' -H "Content-Type: application/json"
curl -X POST $URL/items -d '{"itemName":"Nail Cutter", "itemPrice":2.5}' -H "Content-Type: application/json"
```

4. 在瀏覽器中執行 `$URL/items`，測試 GET。您也可以從指令列執行 `curl`

```
curl -X GET $URL/items
```

5. 測試刪除：現在請執行下列指令，嘗試刪除項目。視需要變更 `item-id` 的值。

```
curl -X DELETE $URL/items/1
```

This throws an error message

```
{"message": "Could not delete Item with id=[object Object]"}
```

找出並修正問題

1. 應用程式正在偵錯模式下執行。因此請使用中斷點找出問題。以下提供幾項訣竅：
 - 我們發現 DELETE 有問題，因為它未傳回預期結果。因此您會在 `itemcontroller.js -> exports.delete` 方法中設定中斷點。
 - 逐步執行並觀察每個步驟的變數，即可在左側視窗中查看本機變數的值。
 - 如要觀察特定值 (例如 `request.params`)，請將這個變數新增至「監看」視窗。
2. 請注意，指派給 `id` 的值是 `undefined`。變更程式碼以修正問題。

修正後的程式碼片段如下所示。

```
// Delete a Item with the specified id in the request
exports.delete = (req, res) => {
  const id = req.params.id;
```

3. 重新啟動應用程式後，請再次嘗試刪除，確認問題是否解決。

4. 按一下偵錯工具列中的紅色方塊



，停止偵錯工作階段。

步驟 7 · 約 0 分鐘

7. 清除所用資源

恭喜！在本實驗室中，您從頭建立新的 Node.js 應用程式，並設定該應用程式，以便在容器中以熱部署模式運作。然後，您按照傳統應用程式堆疊中的相同開發人員流程，將應用程式部署至遠端 GKE 叢集並進行偵錯。

完成實驗室後，請執行下列清理作業：

1. 刪除實驗室中使用的檔案

```
cd ~ && rm -rf mynodejsapp && rm -f setup.sh
```

2. 刪除專案，移除所有相關基礎架構和資源
-